

Базовые методы обучения моделей

Паточенко Евгений
НИУ ВШЭ

План занятия

- Переобучение и недообучение
- Регуляризация в линейной регрессии
- Кросс-валидация
- Подбор гиперпараметров

Напоминание

Линейная регрессия

Базовая модель машинного обучения и статистики, используемая для оценки зависимости между одной зависимой переменной (целевой) и одной или несколькими независимыми переменными (признаками), при этом целевая переменная — непрерывная величина

Общий вид уравнения линейной регрессии

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n + \varepsilon, \text{ где}$$

- y — зависимая (целевая) переменная
- $x_1, x_2, \dots, x_n$ — независимые переменные (признаки)
- β_0 — свободный член (intercept)
- $\beta_1, \beta_2, \dots, \beta_n$ — коэффициенты (веса) модели
- ε — ошибка модели (ошибка прогноза)

Переобучение и недообучение

Переобучение (overfitting) – явление, при котором ошибка модели на объектах, не участвовавших в обучении, оказывается существенно выше, чем ошибка на объектах, участвовавших в обучении.

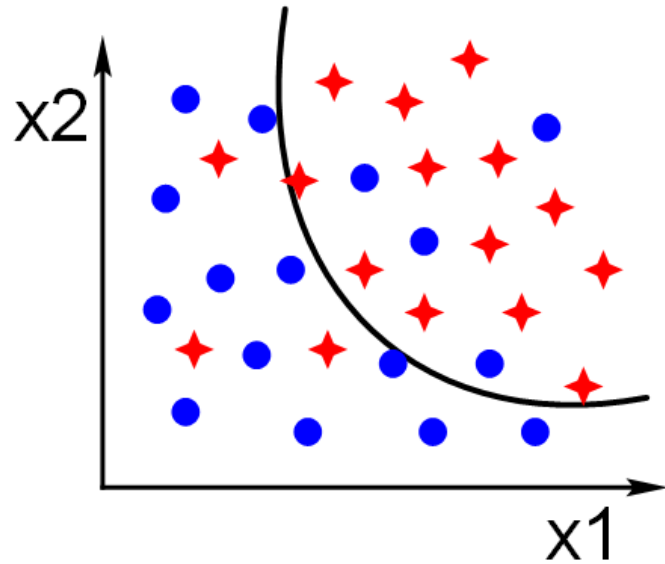
Переобучение возникает при использовании слишком сложных моделей, либо при слишком долгом процессе обучения, либо при неудачной обучающей выборке.

Недообучение (underfitting) – явление, при котором ошибка обученной модели оказывается слишком большой.

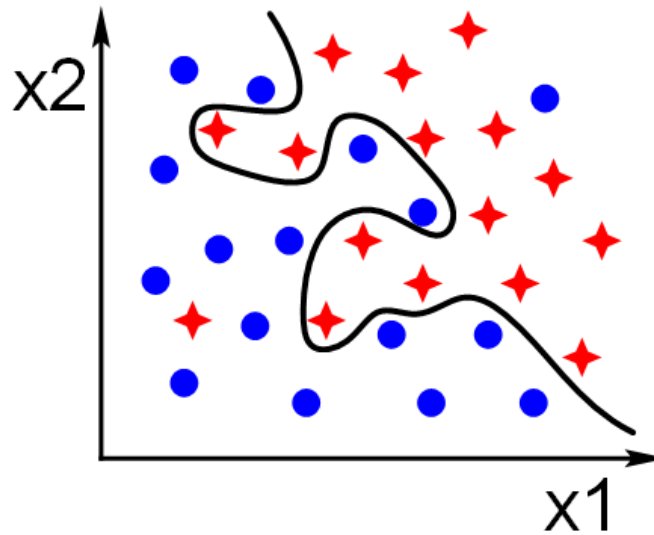
Недообучение возникает при использовании слишком простых моделей, либо при прекращении процесса обучения до достижения состояния с достаточно малой ошибкой, либо при неудачной обучающей выборке.

Переобучение в классификации

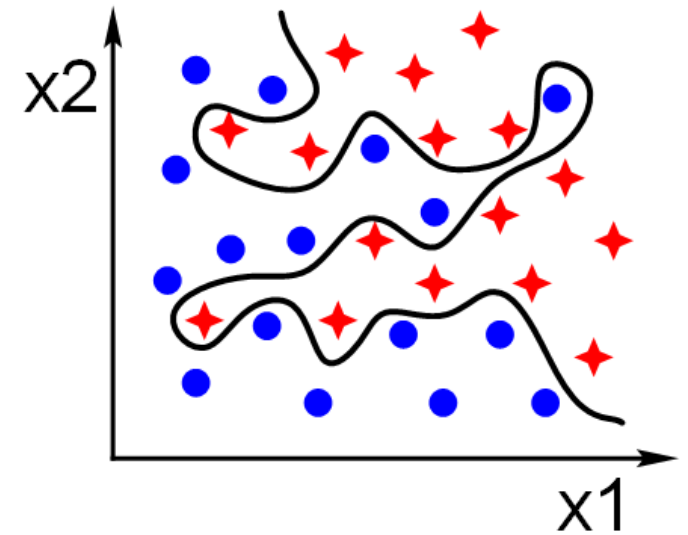
Недообучение



Оптимум

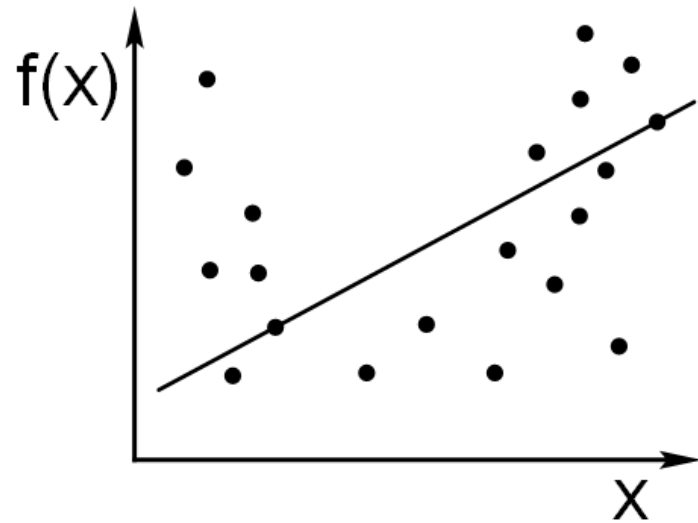


Переобучение

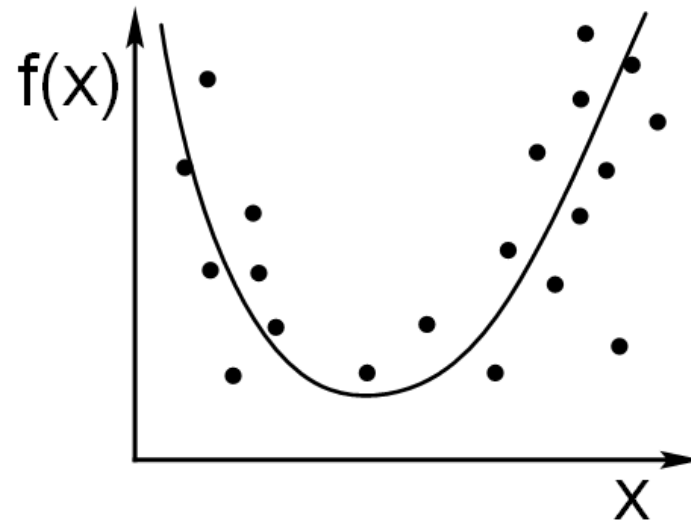


Переобучение в регрессии

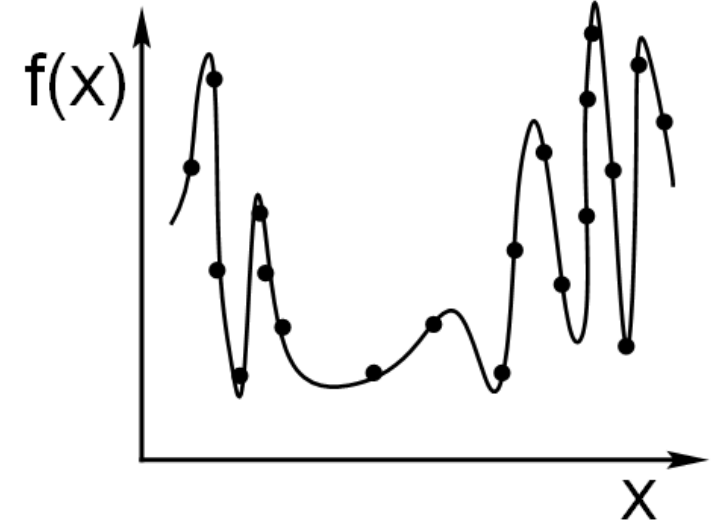
Недообучение



Оптимум



Переобучение



Регуляризация

Зачем нужна?

- При сильной корреляции признаков веса при них теряют физический смысл, а задача оптимизации $Q(\beta) \rightarrow \min$ может иметь бесконечное число решений
- Возможна парадоксальная ситуация: признак, который должен повышать целевую переменную, получает отрицательный вес
- Это делает модель неточной и неинтерпретируемой
- Неадекватность знаков или величины весов — верный признак **мультиколлинеарности** (сильной линейной зависимости между признаками)

Регуляризация

Зачем нужна?

При регуляризации мы модифицируем функцию потерь, добавляя штраф за слишком большие веса

Регуляризация

Зачем нужна?

При регуляризации мы модифицируем функцию потерь, добавляя штраф за слишком большие веса

В общем виде это можно записать так:

$$Q_{\lambda}(\beta) = Q(\beta) + \lambda R(\beta),$$

где λ — коэффициент регуляризации (гиперпараметр, который отражает степень силы штрафа), а $R(\beta)$ — регуляризатор

Регуляризация

Зачем нужна?

При регуляризации мы модифицируем функцию потерь, добавляя штраф за слишком большие веса

В общем виде это можно записать так:

$$Q_{\lambda}(\beta) = Q(\beta) + \lambda R(\beta),$$

где λ — коэффициент регуляризации (гиперпараметр, который отражает степень силы штрафа), а $R(\beta)$ — регуляризатор

λ подбирают на валидационной выборке по логарифмической шкале (например, от $1e-2$ до $1e+2$)

Регуляризация

L2-регуляризация (Ridge)

$$R(\beta) = \|\beta\|_2 = \sum_{i=1}^n \beta_i^2$$
$$Loss = MSE + \lambda \sum \beta_i^2$$

L1-регуляризация (Lasso)

$$R(\beta) = \|\beta\|_1 = \sum_{i=1}^n |\beta_i|$$
$$Loss = MSE + \lambda \sum |\beta_i|$$

Регуляризация

L2-регуляризация (Ridge)

Использует сумму квадратов весов, то есть L2-норму

Штрафует большие значения весов, заставляя их приближаться к нулю, но не зануляет их полностью. Уменьшает влияние шума в данных на модель

В библиотеке scikit-learn реализована в виде класса Ridge ([ссылка на документацию](#))

Регуляризация

L1-регуляризация (Lasso)

Использует сумму модулей весов, то есть L1-норму

Штрафует все веса одинаково, тем самым зануляя наименее важные. Успешно используется для отбора признаков

В библиотеке scikit-learn реализована в виде класса Lasso ([ссылка на документацию](#))

Регуляризация

ElasticNet

Комбинированный подход к регуляризации, сочетающий механизмы Lasso и Ridge

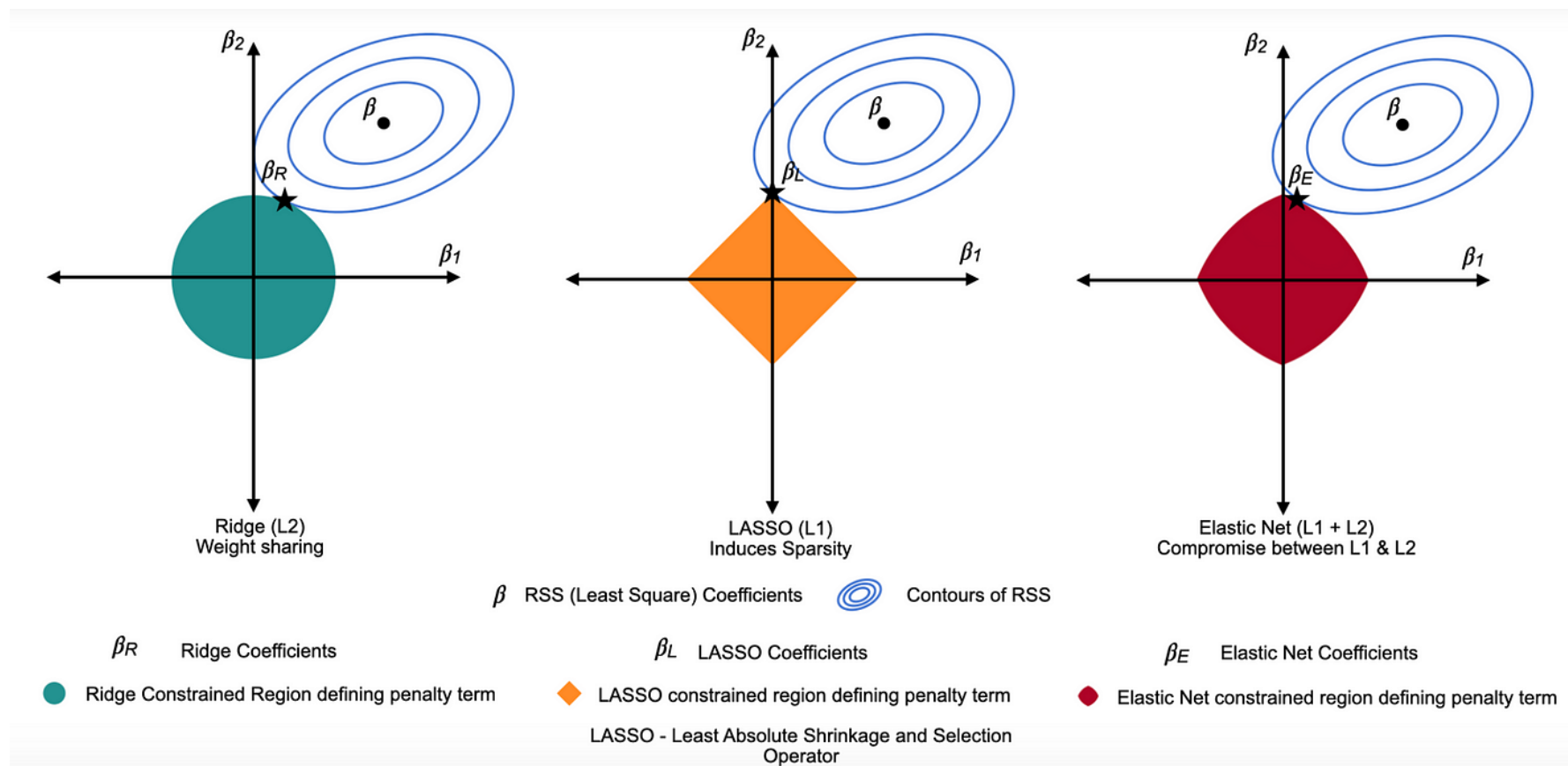
$$R(\beta) = \|\beta\|_1 + \|\beta\|_2 = \sum_{i=1}^n |\beta_i| + \sum_{i=1}^n \beta_i^2$$
$$Loss = MSE + \lambda_1 \sum |\beta_i| + \lambda_2 \sum \beta_i^2$$

Совмещает плюсы L1 и L2: отбирает признаки и сглаживает веса

В библиотеке scikit-learn реализована в виде класса ElasticNet
([ссылка на документацию](#))

Регуляризация

Сравнение методов регуляризации



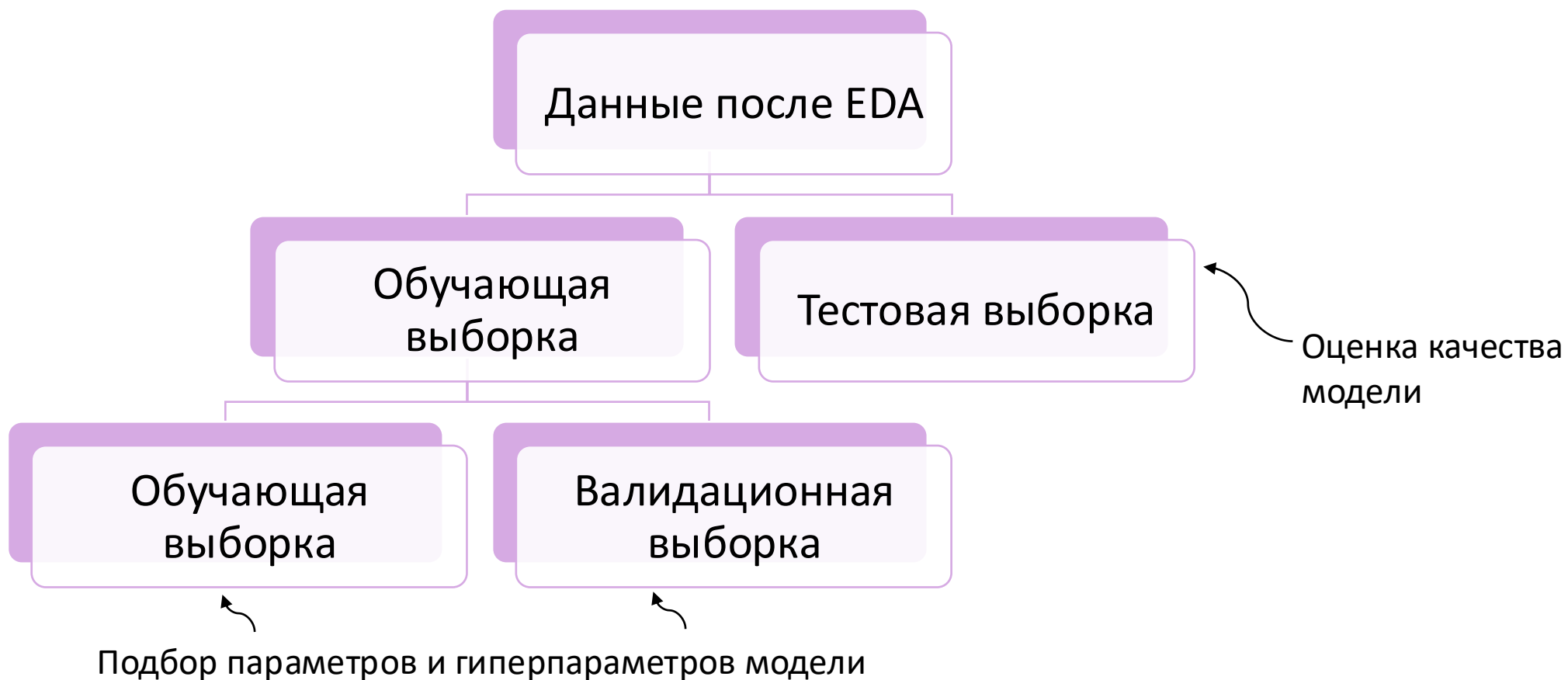
Регуляризация

Плюсы и минусы регуляризации

- + Повышает устойчивость весов в случае мультиколлинеарности
- + Повышает обобщающую способность алгоритма
- + Снижает риск переобучения
- Высокие вычислительные затраты на кросс-валидацию для подбора силы регуляризации λ

Напоминание

Разбиение данных на обучающую и тестовую выборку



Кросс-валидация

Определение

Кросс-валидация — это процедура для оценки качества работы модели

Наиболее часто используемый механизм — **k-Fold CV**: фиксируется тестовая выборка, а обучающая — разделяется на k равных частей (фолдов)

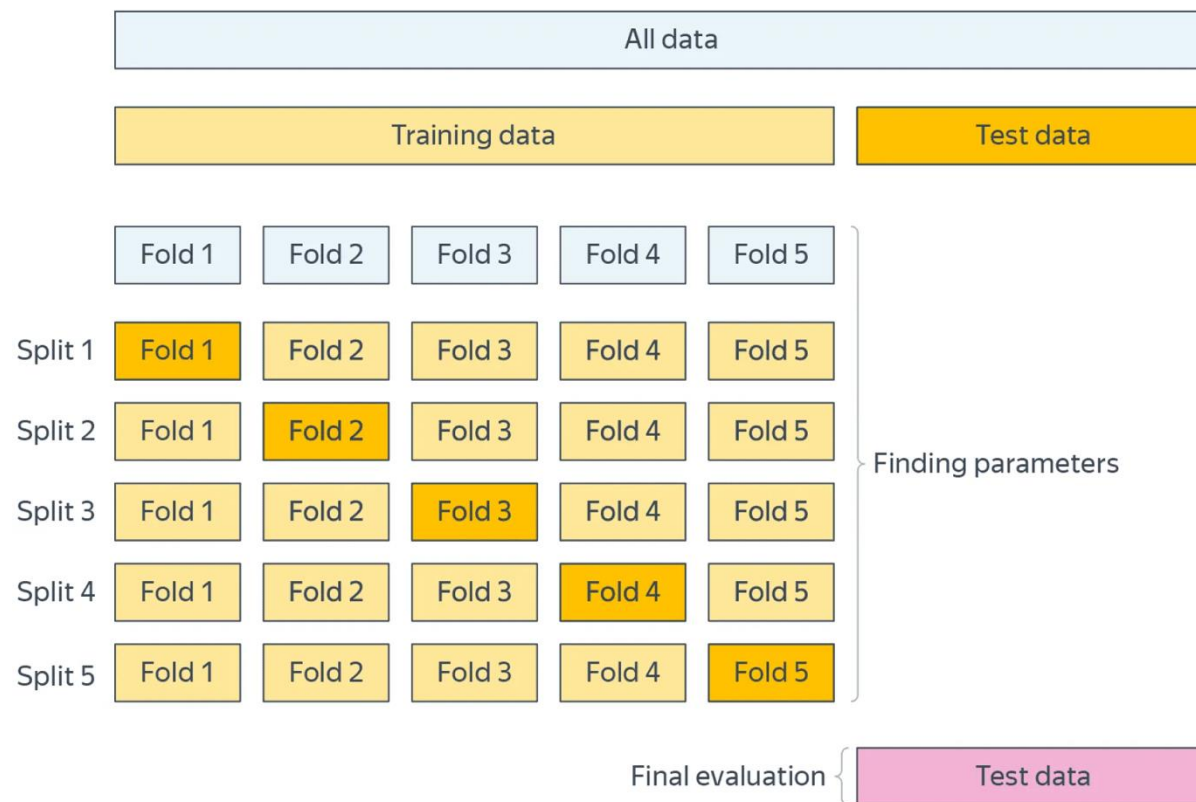
На каждом шаге одна часть используется как тестовая, остальные $k-1$ — для обучения

Процесс повторяется k раз с последовательной сменой валидационной части

Итоговое качество модели — среднее значение метрики по всем фолдам, либо по отложенной выборке

Кросс-валидация

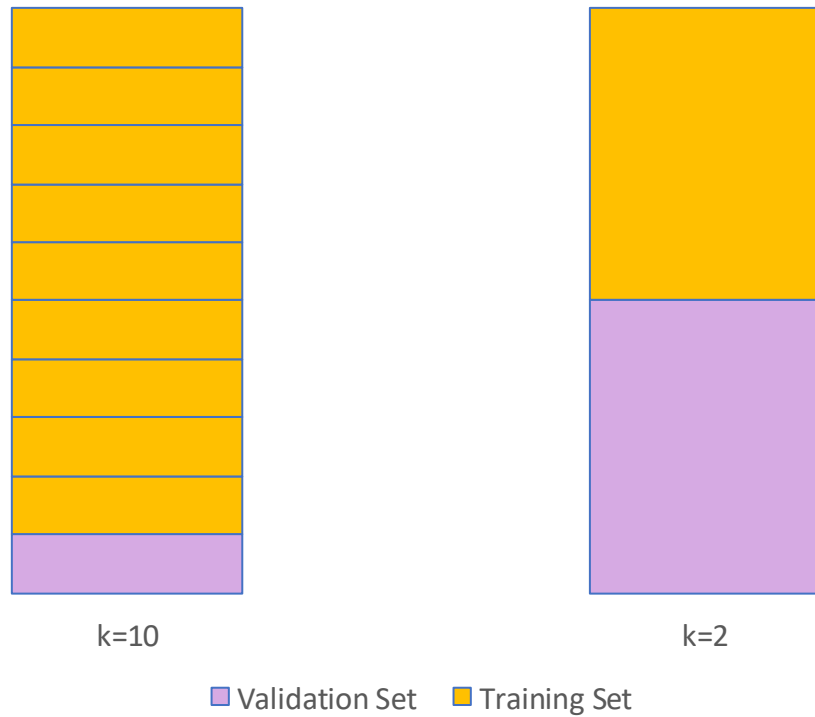
k-Fold CV



Рекомендуем ознакомиться с материалом о кросс-валидации и ее реализации в [scikit-learn](https://scikit-learn.org/)

Кросс-валидация

Выбор количества блоков в k-Fold CV

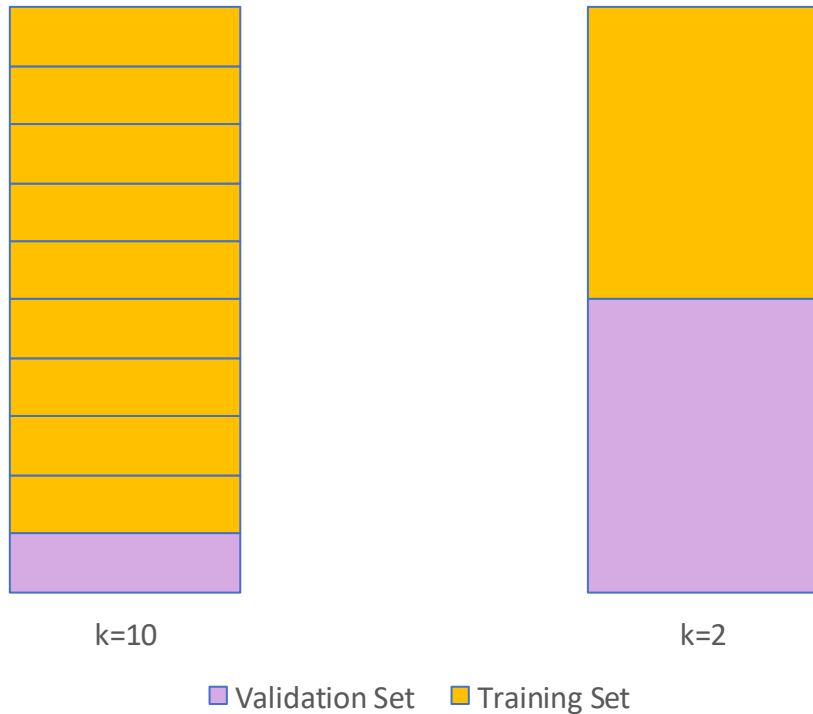


Проблемы при маленьком k?

Проблемы при большом k?

Кросс-валидация

Выбор количества блоков в k-Fold CV



При маленьком k оценка может быть пессимистично занижена из-за маленького размера тренировочной части

При большом k оценка может иметь большую дисперсию из-за маленького размера валидационной части и долго обучать много моделей

Кросс-валидация

Stratified k-Fold

Для разбиения на фолды используется стратификация, то есть каждый фолд содержит примерно такое же соотношение классов, как и все исходное множество.

Полезен для использования с датасетами с дисбалансом классов, когда при обычном случайном разбиении (random split) некоторые фолды могут содержать слишком мало экземпляров некоторых классов или не содержать их вовсе.

[Реализация в scikit-learn](#)

Гиперпараметры и параметры модели

- **Параметры** модели — величины, настраиваемые по обучающей выборке. Например, веса $\mathbf{w}$ в линейной регрессии.
- **Гиперпараметры** модели — величины, контролирующие процесс обучения. Они не могут быть настроены по обучающей выборке. Например, коэффициент α в регуляризации или η шаг обучения в градиентном спуске.

Как подбирать гиперпараметры?

Как правило, подбор гиперпараметров происходит по валидационной выборке. Но при таком подходе получается, что валидация начинает играть роль обучающего множества только для множества гиперпараметров.

Чтобы такого не происходило, исходное обучающее множество данных делят на три подмножества:

- **Train** — основное пространство данных, на которых модель *учится* (происходит поиск параметров модели)
- **Validation** — пространство данных для калибровки модели (замер качества алгоритмов с различными значениями гиперпараметров)
- **Test** — финальный замер качества модели на данных, которые никогда не участвовали в обучении или настройке

Подбор гиперпараметров

- **Grid Search** — перебор по сетке
 - Для каждого гиперпараметра фиксируется несколько значений
 - Последовательно перебираются все возможные комбинации значений гиперпараметров, на каждой из комбинаций обучается и тестируется модель
 - Выбирается модель с наилучшим качеством

Минус метода: если комбинаций значений слишком много, подбор будет длиться неразумно долго

[Реализация в scikit-learn](#)

Подбор гиперпараметров

- **Random Search**

- Для каждого гиперпараметра задается распределение, из которого семплированием выбираются значения
- Последовательно перебираются отобранные случайные комбинации значений, на каждой из комбинаций обучается и тестируется модель
- Выбирается модель с наилучшим качеством

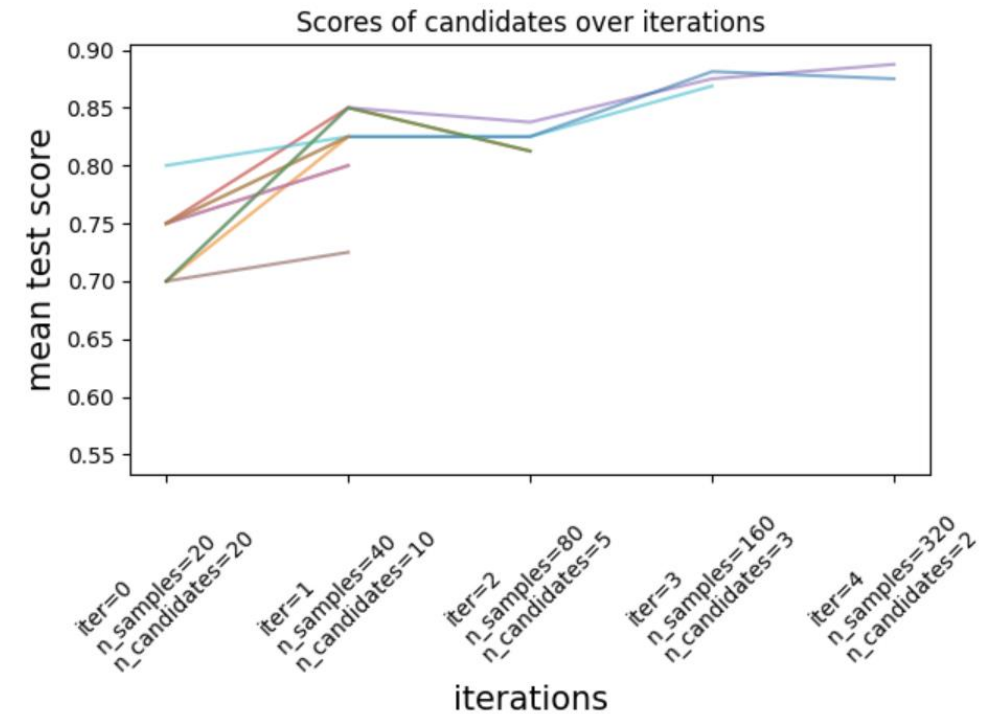
Преимущество: при достаточно большом количестве перебираемых значений RS может за то же число итераций, что GS, рассмотреть более разнообразные значения. Таким образом, мы оптимизируем поиск без риска пропустить хорошую комбинацию

[Реализация в scikit-learn](#)

Подбор гиперпараметров

- **Последовательный халвинг (Successive halving, SH)** — все комбинации параметров оцениваются с небольшим количеством ресурсов на первой итерации. На вторую итерацию с увеличенными ресурсами проходят только те комбинации, которые показали лучшие результаты. Процесс повторяется до нахождения наилучшей комбинации

Реализация в [scikit-learn по полной сетке](#)
и [с семплированием](#)



Подбор гиперпараметров

Другие методы

- Байесовская оптимизация
- Tree-structured Parzen Estimator (TPE)
- Population Based Training (PBT)
- Hyperband
- и т. д.

В простых случаях используются Grid Search и Random Search

Подбор гиперпараметров

Основные библиотеки для подбора гиперпараметров

- [Scikit-learn](#)
- [Hyperopt](#)
- [Optuna](#)